

The AI Engineer Roadmap

From Prerequisites to Job-Ready
Track A (Job-Ready) + Track B (Depth)

[@aiwithapoorvaa](#)

A free, resource-backed roadmap for AI Engineers who build with existing LLMs — not for training models from scratch.

The AI Engineer Roadmap (Part 2) — From Prerequisites to Job-Ready

Compiled by @aiwithapoorvaa

This is the continuation of the **Prerequisites** post. If you haven't gone through that one yet, do that first — this roadmap assumes you already know basic Python syntax, basic Git, basic API concepts, and have started your logic-building practice.

This roadmap is for AI Engineers who build applications using existing LLMs (GPT, Claude, Gemini, etc.) — not for people trying to become ML/Data Scientists who train models from scratch. If you later want to go deeper into ML/DL theory, that's what Track B is for. But you do not need it to get hired and start working as an AI Engineer.

How this roadmap is structured — read this before anything else

Most roadmaps online are one long list that takes 12-14 months before you're "ready" to apply anywhere. That's demotivating and, honestly, not true. You do not need to know how to build a neural network from scratch to get hired as an AI Engineer who builds RAG systems and agents on top of GPT/Claude APIs.

So this roadmap is split into two tracks:

- **TRACK A — Job-Ready Track.** Everything you actually need to build real LLM applications (RAG, agents, tool calling, MCP), deploy them, and start applying for jobs / freelance work. Do this first, in order.
- **TRACK B — Depth Track.** Full ML fundamentals, PyTorch, deep evaluation/LLMOps, system design, advanced backend. Do this **in parallel with applying and interviewing**, not before it. Some of Track B you may never need depending on the kind of AI engineering role you want — that's fine.

Rule of thumb: Track A gets you hired. Track B makes you excellent. Don't let Track B block you from starting Track A projects and applying to jobs.

At a realistic pace of 12-15 hours/week, Track A takes roughly **5-6 months** if you've already done the prerequisites properly (this went up slightly from the earlier version of this roadmap — I added three sections that were genuinely missing: evaluation, security, and local models. More on why below). Track B is ongoing after that — you keep layering it in.

One honest note before you start: I've trimmed every resource here down to the shortest one I could find that still genuinely teaches the concept — nobody needs a 19-hour video when a 1-hour one covers what you actually need at this stage. But shorter resources ≠ less work. You'll still have to write the code yourself, get stuck, debug, and rebuild things two or three times before they click. That part cannot be compressed. What this roadmap compresses is wasted time, not effort.

TRACK A — Job-Ready Track

| A1. Python — the rest of it (2-3 weeks)

Your prerequisites doc covered syntax, data structures, and async. Here's what was still missing and that you actually need before touching backend/LLM work:

Topics to cover: - Object-Oriented Programming — classes, `__init__`, inheritance, class vs instance variables, dunder/magic methods - Exception handling — `try/except/finally`, raising custom exceptions - File handling — reading/writing files, `pathlib` - Virtual environments and package management (use `uv`, it's faster and cleaner than pip) - Type hints - Decorators and context managers (`with` statement) — at least conceptually, you'll see these constantly in FastAPI and LLM SDKs - JSON handling in Python (`json.loads`, `json.dumps`) — you will do this every single day working with LLM APIs - `pytest` basics — writing your first tests

Resources: - 📖 Read: [Real Python — OOP in Python 3](#) - 🎥 Watch: [Corey Schafer — Python OOP Tutorial Series](#) (classes, inheritance, dunder methods, property decorators) - 🎥 Watch: [Corey Schafer — Try/Except Error Handling](#) - 🎥 Watch: [Corey Schafer — Python Programming Beginner to Advanced playlist](#) (covers file handling, venv, JSON, requests — search his channel for each topic individually, all free) - 📖 Read: [Real Python — Python Type Checking Guide](#) - 🎥 Watch: [freeCodeCamp — Pytest Tutorial: How to Test Python Code](#) (covers first tests, fixtures, mocking)

Checkpoint: You can write a Python script with a class-based structure, proper error handling, file I/O, and at least 2-3 passing pytest tests — with no hardcoded values.

| A2. Git + Linux/CLI basics (1 week, parallel with A1)

You already have the Git command list from the prerequisites doc. Add basic Linux command-line literacy — non-negotiable if you're going to work with VMs, servers, or deployment (which you will).

Additional topics: `cd`, `ls`, `pwd`, `mkdir`, `rm`, `cp`, `mv`, `cat`, `grep`, `chmod`, `ps`, `kill`, `top`, piping (`|`), redirects (`>`, `>>`)

Resources: - 📖 Read (official, bookmark this): [GitHub Docs — Git Cheat Sheet](#) - 🎥 Watch: [freeCodeCamp — Git & GitHub Crash Course for Beginners](#) (branching, merging, stashing, rebase, pull requests) - 📖 Read: [freeCodeCamp — Learn How to Use Git and GitHub: A Beginner-Friendly Handbook](#)

Checkpoint: You can navigate a Linux filesystem, check running processes, and comfortably push/pull/branch/merge in Git without googling every command.

| A3. Backend basics — just enough (3-4 weeks)

You need enough backend to expose your AI applications as real APIs and store data — not a full backend engineering mastery.

Topics: - FastAPI — routes, path/query params, request bodies, Pydantic models, response models - REST concepts (you have the basics from prerequisites — now apply them by building, not just reading) - SQL basics — SELECT, WHERE, JOIN, GROUP BY (not full PostgreSQL administration) - Environment variables and `.env` files - Basic error handling and status codes in an API context

What NOT to go deep on yet (moved to Track B): JWT authentication internals, Redis, WebSockets, full ORM theory. You need to know these exist, not master them yet.

Resources: - 🎥 Watch: [freeCodeCamp — FastAPI Course for Beginners](#) (1-hour crash course — installation, path/query params, POST/PUT/DELETE) - 📖 Read (official): [FastAPI documentation](#) — genuinely one of the best-written docs of any framework, use it as your reference - 🎥 Watch: [freeCodeCamp — SQL Tutorial: Full Database Course for Beginners](#) (tables, keys, joins, nested queries, ER diagrams)

Build: A small FastAPI CRUD app (e.g., a notes API or task tracker) with at least 4 endpoints and a SQLite/PostgreSQL backend.

Checkpoint: You can build and test a working REST API locally with proper request validation.

| A4. Minimum viable ML/math literacy — NOT ML engineering (1-2 weeks)

This is deliberately light. You need to *understand* what's happening conceptually so you can reason about LLM behavior and speak intelligently in interviews — you do **not** need to train models or derive backpropagation by hand at this stage. That's Track B, and it's genuinely optional depending on the AI engineering path you choose.

Topics: - What vectors and embeddings are, and why "similarity" works mathematically (dot product / cosine similarity intuition) - What attention/transformers do at a conceptual level — no math derivation needed - Basic probability — enough to understand "the model predicts a distribution over the next token"

Resources: - 📖 Read: [Towards AI — All Linear Algebra Concepts You Need for ML](#) (from your prerequisites doc — this is enough linear algebra for now) - 🎥 Watch: [3Blue1Brown — But what is a GPT? Visual intro to transformers](#) (the single best conceptual explainer of how LLMs actually work, no math background needed) - 🎥 Watch (optional, if you want a bit more depth): [3Blue1Brown — Attention in Transformers, visually explained](#) - 📖 Read: [DEV Community — Beginner's Guide: Statistics and Probability in ML](#) (from prerequisites doc)

Checkpoint: You can explain, in plain English, what an embedding is and why attention lets a model "look at" relevant words. You do not need to write this code from scratch.

| A5. LLM application engineering (4-5 weeks)

This is where things get exciting — this is the actual "AI Engineer" part.

Topics: - Calling LLM APIs (OpenAI, Anthropic, etc.) - System prompts, prompt design, few-shot prompting - Temperature and other generation parameters - Structured outputs / JSON schema enforcement - Function calling / tool calling - Streaming responses - Token usage, cost, and latency basics - Model selection — when to use a smaller/cheaper model vs a frontier one - **Managing multi-turn conversation history and context windows** — what happens when a conversation gets too long, how to decide what to trim, and why trimming the wrong turn silently breaks your bot (this is a real, common bug — not a theoretical one)

Resources: - 📖 Read (official, essential): [Anthropic — Prompt Engineering Overview](#) - 🎥 Do: [Anthropic's Interactive Prompt Engineering Tutorial \(GitHub\)](#) — free, hands-on, 9 chapters with exercises. This is the single best resource for learning prompting properly. - 📖 Read: [Anthropic Engineering — Effective Context Engineering for AI Agents](#) (how to structure system prompts, tools, and message history well — this also directly covers context window and conversation-trimming strategy) - 📖 Read (official docs, use as reference): OpenAI's [Function Calling guide](#) and Anthropic's [Tool Use guide](#)

Build: 1. A chatbot with streaming responses 2. A structured data extractor (e.g., extract name/date/amount from unstructured text into JSON) 3. An assistant that calls at least one real external tool (weather API, search, or a small database)

Checkpoint: You can build an LLM app that reliably returns structured, validated output and uses at least one tool call, and you can explain what your app does when the conversation exceeds the model's context window.

| A6. Lightweight LLM evaluation & prompt testing (1 week)

This is not optional, and it's not the same as the full LLMops section in Track B. This is the basic habit of proving a prompt change actually helped, instead of eyeballing 3 outputs and shipping. Every interviewer will ask some version of "how do you know your system works" — this section is how you get a real answer instead of "it felt better."

Topics: - Why LLM outputs can't be tested like normal code (non-deterministic, no exact-match assertions) - Building a small test set of real inputs + expected behavior (10-20 cases is enough to start) - Deterministic checks (does it contain X, is it valid JSON) vs. model-graded checks (does this sound right) - Running the same test set before/after a prompt change and comparing results side by side - Comparing two models or two prompts on the same test set

Resources: - 🛠 Do: [Promptfoo — official documentation and quickstart](#) (open-source, free, the standard tool for this — "test-driven prompt engineering instead of trial-and-error") - 📖 Read: [DataCamp — Promptfoo Tutorial: A Hands-On Guide to LLM Evaluation](#) (walks through building a real eval suite from scratch, step by step) - 📖 Read: [GitHub — Promptfoo Quickstart Guide](#) (a shorter, code-first version if you just want to see it working fast)

Build: Take the structured data extractor or chatbot you built in A5, write a 15-case test set covering normal inputs and edge cases, and run it through Promptfoo before and after you change the prompt. Save both result sets.

Checkpoint: You can show, with numbers, that a prompt change you made actually improved (or didn't improve) your app's outputs — not "I think it's better now."

| A7. LLM security basics: prompt injection & guardrails (4-5 days)

You will build this into every real LLM application you ship — this is not an advanced or optional topic. Any app that reads untrusted user input, or retrieves content from documents/websites/databases and feeds it to an LLM, is exposed to this. If you're building a RAG app right after this section, you need this first, not after.

Topics: - Prompt injection: direct (user types a malicious instruction) vs. indirect (malicious instructions hidden inside a retrieved document or webpage the LLM reads) - Why "just tell the model to ignore instructions in user input" is not a real fix - Basic mitigations: treating all retrieved/external content as untrusted, input/output validation, limiting what tools an LLM can call without human approval, least-privilege tool permissions - Why this matters more in agentic systems — an LLM that can send emails or call APIs is a much bigger risk than one that only outputs text

Resources: - 📖 Read (official, essential): [OWASP Top 10 for LLM Applications](#) (the industry-standard reference — start with LLM01: Prompt Injection) - 📖 Read: [Check Point — What Is Prompt Injection?](#) (clear breakdown of direct vs. indirect injection with real attack patterns) - 📖 Read: [Promptfoo — OWASP LLM Top 10 red-teaming guide](#) (shows you how to actually test your own app for these vulnerabilities using the tool you just learned in A6)

Build: Take your assistant from A5 (the one with tool calling) and deliberately try to break it — write 5 adversarial inputs designed to make it ignore its system prompt or misuse a tool. Then add at least one real mitigation and confirm it holds.

Checkpoint: You can explain the difference between direct and indirect prompt injection with a real example, and you've stress-tested at least one of your own apps against it — this is exactly the guardrail work goes into any production agent or chatbot before it ships.

| A8. RAG — Retrieval-Augmented Generation (4-5 weeks)

This is core to most real AI engineering jobs right now — it's one of the most commonly requested skills in AI engineering job postings.

Full pipeline to learn: Document parsing → chunking → embeddings → vector database → retrieval → reranking → LLM answer → citations

Topics: - **Document parsing for messy real-world files — don't skip this.** Most tutorials assume a clean PDF. Real input is scanned contracts, Excel exports, DOCX files with embedded tables, low-quality scans. Learn the difference between basic text extraction and proper layout-aware parsing/OCR. - Chunking strategies and chunk size/overlap tradeoffs - Embedding models - Vector similarity and vector databases (Chroma, Pinecone,

FAISS, etc.) - Hybrid search, metadata filtering - Reranking, query rewriting - Citations and grounding - Basic retrieval evaluation — how do you know if a bad answer is a retrieval problem or an LLM problem? (use the eval habits from A6 here)

Resources: - 📺 Watch: [freeCodeCamp / LangChain — RAG From Scratch](#) (taught by a LangChain engineer — indexing, retrieval, generation, query translation methods) - 📺 Watch: [Krish Naik — Complete RAG Crash Course with LangChain in 2 Hours](#) (great if you prefer an Indian-instructor teaching style, code on GitHub included) - 📖 Read: [Machine Learning Mastery — A Practical Guide to Building Local RAG Applications with LangChain](#) - 📖 Read (official, use as reference): [LangChain — RAG documentation](#) - 📖 Read: [BuildRAG — Parsing PDF Documents](#) (hands-on comparison of Unstructured vs. Docling for tables, scans, and complex layouts — this is the part most RAG tutorials skip and where most real RAG bugs actually live)

Build: A "chat with your documents" app — PDF upload (include at least one messy/scanned file, not just a clean one), chunking, vector search, cited answers, and basic conversation history.

Checkpoint: Given a bad answer from your RAG app, you can diagnose whether it's a parsing failure, a retrieval failure, a context-construction failure, or an LLM failure — that layered debugging instinct is what separates a working RAG demo from a production-grade one.

A9. AI agents and tool calling (3-4 weeks)

Topics: - The agent loop: goal → reasoning → tool selection → execution → observation → next action - Deterministic workflows vs. autonomous agents (start with the former) - State management, short-term memory - Retries, timeouts, error handling, human-approval steps - Basic guardrails

Important distinction to internalize:

Workflow	Agent
Developer defines the steps	Model chooses actions
More predictable	More flexible
Easier to test	Harder to evaluate
Often better for business processes	Better for open-ended tasks

Resources: - 📖 Read: [aibuilderclub — AI Agents in 2026: Build, Deploy, and Scale](#) (a genuinely well-structured free guide covering the agent loop, frameworks vs. raw API, memory, and multi-agent systems) - 📺 Watch: [Tool Calling — OpenAI Agents Tutorial Ep. 4](#) (practical, code-along style) - 📖 Read: [MachineLearningMastery — The Roadmap to Mastering Tool Calling in AI Agents](#)

Build: A deterministic workflow (not a fully autonomous agent) that calls 2-3 tools and produces a structured report — e.g., a research assistant that searches, pulls data, and summarizes.

Checkpoint: You can explain to an interviewer, with a real example, when you'd choose a deterministic workflow over an autonomous agent — and why that matters for production reliability.

A10. MCP — Model Context Protocol basics (1-2 weeks)

Learn this **after** you understand tool calling — MCP is a standardized way of exposing tools to AI apps, not a new concept on its own.

Topics: - What MCP is and why it exists (standardizing how AI apps connect to external tools/data) - MCP clients, servers, tools, resources, prompts - Connecting an AI app to an existing MCP server - Building a very simple MCP server (one tool)

What NOT to go deep on yet (Track B): Full auth/permissions architecture, trust boundaries, production security hardening for MCP servers.

Resources: - 📺 Watch: [freeCodeCamp — Intro to MCP Servers: Model Context Protocol with Python](#) (build 3

real MCP servers hands-on using FastMCP) - 📖 Read (official, essential): [Anthropic — Introduction to Model Context Protocol course](#) - 📖 Read: [freeCodeCamp — How the Model Context Protocol Works](#) - 📖 Read (deeper reference when ready): [Microsoft — MCP for Beginners \(open-source curriculum\)](#)

Build: A simple MCP server exposing one real tool (e.g., a calculator or a document-search function), and connect an agent to it.

Checkpoint: You can explain the client → MCP client → MCP server → external tool flow, and you've built and tested one server yourself.

| A11. Local & open-source model tooling (4-5 days)

A real, current market skill — not a niche side-topic. Not every client wants to pay per-token on a hosted API forever, especially cost-conscious teams in the Indian market. Knowing how to run an open-weight model locally (Llama, Mistral, etc.) makes you useful in situations where a fully API-dependent engineer isn't.

Topics: - What Ollama is and why teams use local/open-weight models (privacy, cost, offline use, no per-token billing) - Pulling and running a model locally, basic Modelfile customization - Using a local model as a drop-in replacement in code you'd otherwise point at OpenAI/Anthropic (same API shape, different base URL) - Realistic tradeoffs — when a local open-weight model is good enough, and when it isn't

Resources: - 🎥 Watch: [freeCodeCamp — Local AI Development with Ollama](#) (installing, customizing models, REST API and Python library usage) - 📖 Read: [freeCodeCamp — How to Run and Customize LLMs Locally with Ollama](#) (shorter written version if you'd rather read than watch — installation through Modelfile customization in about 15-20 minutes)

Build: Take one small piece of your A5 or A8 project and swap the hosted LLM API call for a local Ollama model — confirm it still works, and note where the quality gap actually shows up.

Checkpoint: You've run a model entirely on your own machine with no API key and no internet dependency, and you can explain to an interviewer when you'd recommend a local model over a hosted API for a client.

| A12. Deploy something — imperfect is fine (1-2 weeks)

This section is deliberately scoped to just getting one thing live — the goal here is making sure you actually ship something with a public URL, not achieve full production-grade infrastructure.

Topics: - Docker basics — images, containers, Dockerfile - Deploying one project to a VM or a simple cloud platform - Environment variables in production (not hardcoded secrets)

What NOT to go deep on yet (Track B): Full CI/CD pipelines, Kubernetes, advanced monitoring stacks.

Resources: - 🎥 Watch: [freeCodeCamp — Docker Tutorial for Beginners \(Full Course in 3 Hours\)](#) - 📖 Read: [freeCodeCamp — The Docker Handbook](#)

Build: Take your RAG or agent project from A8/A9 and actually deploy it — even a bare-bones version on a free-tier VM or cloud run service counts.

Checkpoint: You have at least one live, working AI application with a public URL.

| A13. Portfolio + interview prep (ongoing from here)

What to do: - Pick your 2 strongest projects from A5-A12 (RAG app + agent/workflow app are the strongest combo) and polish them - Push everything to GitHub with a real commit history — not one giant commit - Write 1-2 debugging case studies from your own project work — pick a real bug you hit while building your RAG or agent project (an OOM error, a race condition, an auth mismatch, whatever actually broke), and write it up in a Situation → Problem → Design → Breakage → Fix → Outcome format - Start applying now. Don't wait for Track B.

Resources: - 📖 Read: [freeCodeCamp — How to Write a Good README File for Your GitHub Project](#) (your

portfolio projects are only as good as whether a recruiter can understand them in 30 seconds — this is what makes that happen) - 📖 Read: [freeCodeCamp — How to Prepare for a Technical Interview](#) (general technical interview structure, interview types, and what to expect) - 🎥 Watch: [freeCodeCamp — SQL Interview Questions Solved \(Apple, LinkedIn, Meta, Uber, Amazon, Google\)](#) (SQL itself is covered back in A3 — this is here specifically for the real company interview-question walkthroughs, useful since SQL still comes up in a lot of AI engineering interviews)

Checkpoint: You have a GitHub with 2+ real, working, documented AI projects — each with a clear README — and can talk through your debugging decisions in an interview using specifics — not "I used AI to build it," but "here's the exact bug, here's how I found it, here's the fix."

TRACK B — Depth Track (parallel with applying / after landing interviews)

Do NOT feel behind if you haven't touched this yet. This is what separates a good AI engineer from a great one — but none of it blocks you from getting hired.

B1. Full ML fundamentals

Regression, classification, clustering, cross-validation, bias-variance tradeoff, overfitting/underfitting, feature engineering, data leakage.

- 🎧 Watch: [StatQuest — Machine Learning playlist by Josh Starmer](#) (the best-loved, clearest explainer channel for ML fundamentals, covers everything from linear regression to random forests to gradient boosting)
- 📖 Read: [Analytics Vidhya — A Guide to Complete Statistics for Data Science Beginners](#) (from your prerequisites doc)

B2. PyTorch + transformers from scratch

Only pursue this deeply if you want to move toward more ML-engineering-flavored roles, or a specific job requires it.

- 📖 Read: [Sebastian Raschka — PyTorch in One Hour](#) (exactly what it says — tensors through training neural nets on GPUs, written by a genuinely respected ML educator, in about an hour)
- 🎧 Watch: [AssemblyAI — PyTorch Crash Course: Getting Started with Deep Learning](#) (tensor basics, autograd, linear regression, neural nets, CNNs — a real crash course, not a multi-day commitment)
- If you want the comprehensive, everything-covered version later once you're confident: [freeCodeCamp — PyTorch for Deep Learning & Machine Learning, Full Course](#) (the well-known ~25-hour Daniel Bourke course) — treat this as a reference to revisit by chapter, not a single sitting.

B3. Deep backend

JWT authentication internals, Redis and caching, WebSockets, ORM deep-dive (SQLAlchemy).

- 📖 Read: [TestDriven.io — Securing FastAPI with JWT Token-based Authentication](#) (focused, hands-on, no fluff — you can work through this in an afternoon instead of a weekend)
- 📖 Read: [Better Stack — Authentication and Authorization with FastAPI: A Complete Guide](#) (JWT + SQLAlchemy + database integration in one focused guide)
- If you genuinely want the long-form, everything-in-one-place version later: [freeCodeCamp — Creating APIs with Python, 19-Hour Course](#) covers JWT, PostgreSQL, SQLAlchemy, testing, Docker, and CI/CD end-to-end — but treat it as a reference to dip into by chapter, not something to watch start to finish.

B4. Full evaluation and LLMOps

Golden datasets, LLM-as-judge, retrieval metrics, hallucination rate measurement, observability/tracing, prompt versioning, cost/latency monitoring.

- This space moves fast — when you get here, search for the current documentation from your LLM provider (Anthropic/OpenAI evaluation docs) plus tools like Langfuse or Arize for observability, since best practices here shift every few months.

B5. Multimodal AI

Vision-language models, speech-to-text/text-to-speech, document understanding.

- If you've worked on voice-based products before (STT/TTS pipelines, voicebots), you already have a head

start on this section — that experience transfers directly.

- Otherwise, start with the STT/TTS basics of whichever provider you're most likely to use on the job (OpenAI Realtime API, Azure Speech, ElevenLabs, etc.) and build a small voice-in/voice-out demo before going deeper into vision-language models.

B6. Full MCP security depth

Authentication, permissions, trust boundaries, tool input validation for MCP servers in production.

- 📖 Read: [Microsoft — MCP for Beginners \(GitHub curriculum\)](#) (the fuller modules beyond the basics covered in Track A)

B7. System design for AI applications

Designing a document Q&A platform, a multi-tenant RAG system, an AI customer-support system — thinking about latency, cost, reliability, security, scalability.

- Practice by sketching designs for the systems described in Track A's builds, but at "handle 10,000 users" scale instead of "works on my laptop" scale.

B8. Full cloud/deployment

CI/CD pipelines, secrets management, scaling, queues and background workers, API security in depth.

- 📺 Watch: [freeCodeCamp — Docker and Kubernetes, Full Course for Beginners](#)
-

Quick-reference: what to prioritize if you're overwhelmed

If you only remember one thing from this entire roadmap:

Track A (A1 → A12), build the 2 projects in A13, then start applying immediately. Everything in Track B is deepening, not gatekeeping. A lot of working AI engineers do their jobs well without ever touching half of Track B.

Save this roadmap. Come back to each section as you progress — don't try to read it all in one sitting.

If you have questions on any specific section, drop a comment or DM — that's exactly how this roadmap got made in the first place.

— @aiwithapoorvaa